

Conferencia 1: qué significa que un algoritmo es eficiente

Diseño y Análisis de Algoritmos · Ciencia de la Computación · MatCom · 2026

En este curso, cada vez que aparezca un algoritmo vamos a hacerle las mismas tres preguntas: si es correcto, cuánto cuesta y si se puede hacer mejor. Antes de responderlas hace falta ponerse de acuerdo en qué significa "cuánto cuesta". Esa es la clase de hoy. Al terminar vas a saber en qué modelo de computadora contamos, qué contamos, cómo se escribe el resultado, y por qué un algoritmo que parece rápido puede ser exponencial.

1. Una pregunta con dos respuestas

¿Cuánto cuesta decidir si un número n es primo? El algoritmo que todos conocemos prueba los divisores desde 2 hasta $\sqrt{n}$. Si ninguno divide a n , es primo.

```
def es_primo(n):
    divisiones = 0
    d = 2
    while d * d <= n:
        divisiones += 1
        if n % d == 0:
            return False, divisiones
        d += 1
    return n >= 2, divisiones

for n in [97, 10_007, 1_000_003, 1_000_000_007]:
    primo, divisiones = es_primo(n)
    print(f"{n:>13,} primo: {primo!s:5} divisiones: {divisiones:,}")
```

97	primo: True	divisiones: 8
10,007	primo: True	divisiones: 99
1,000,003	primo: True	divisiones: 999
1,000,000,007	primo: True	divisiones: 31,621

Para n primo hace unas $\sqrt{n}$ divisiones. Crece más despacio que n , así que parece un buen algoritmo. Sin embargo, este algoritmo no puede decidir si un número de 1024 bits es primo antes de que se apague el Sol, y esa operación la hace tu teléfono cada vez que abre una conexión segura, con otro algoritmo. Las dos afirmaciones son ciertas. Al final de la clase vamos a ver por qué, y para eso necesitamos precisar qué es el tamaño de un problema y qué es una operación.

2. Medir en segundos no sirve

La primera idea para medir un algoritmo es programarlo y cronometrarlo. El número que sale depende del procesador, del lenguaje, del compilador, de qué otros programas están corriendo y de la entrada que elegimos probar. Dos personas que miden el mismo algoritmo obtienen números distintos, y ninguno de esos números dice qué va a pasar con una entrada diez veces más grande.

Lo que queremos es una medida que dependa solo del algoritmo y del tamaño de la entrada. Para eso fijamos un modelo de cómputo, una computadora abstracta con reglas precisas sobre qué operaciones existen y cuánto cuesta cada una, y contamos operaciones en ese modelo.

3. La máquina de Turing

Ya conoces un modelo de cómputo: la máquina de Turing. Tiene una cinta infinita dividida en celdas, un cabezal que lee y escribe una celda a la vez y se mueve una posición a la izquierda o a la derecha, y un conjunto finito de estados. Una función de transición dice, según el estado y el símbolo leído, qué escribir, hacia dónde moverse y a qué estado pasar.

La tesis de Church-Turing afirma que todo lo que se puede calcular con un procedimiento mecánico se puede calcular con una máquina de Turing. Por eso es el modelo con el que se define qué es computable.

Para medir costos, en cambio, la máquina de Turing es mala. Para leer la celda i el cabezal tiene que caminar hasta ella, así que acceder a un dato cuesta tanto como la distancia a la que está. Un ejemplo concreto: decidir si una cadena de longitud n es un palíndromo. En cualquier computadora real se compara el primer carácter con el último, el segundo con el penúltimo, y se termina en $O(n)$. Una máquina de Turing de una cinta tiene que ir y volver de un extremo al otro, y Hennie demostró en 1965 que ninguna máquina de una cinta lo resuelve en menos de $\Omega(n^2)$ pasos.

Los modelos razonables se simulan unos a otros con un costo polinomial: una máquina de k cintas que tarda t pasos se simula con una de una cinta en $O(t^2)$ pasos. Eso significa que la frontera entre polinomial y exponencial no depende del modelo, y vamos a usarlo en el Tema 3. Pero el exponente sí depende del modelo, y en este curso nos importa la diferencia entre n , $n \log n$ y n^2 . Necesitamos un modelo más parecido a una computadora.

4. El modelo RAM

El modelo RAM (random access machine) tiene una memoria formada por celdas numeradas $0, 1, 2, \dots$, cada una capaz de guardar un entero. Un programa es una secuencia de instrucciones de este tipo:

- leer o escribir una celda, directamente o con la dirección guardada en otra celda;
- sumar, restar, multiplicar, dividir y comparar enteros;
- saltar a otra instrucción, con o sin condición.

Cada instrucción cuesta 1. El tiempo de un algoritmo con una entrada es el número de instrucciones que ejecuta, y la memoria es el número de celdas que usa. Acceder a la celda 10^9 cuesta lo mismo que acceder a la celda 0, que es lo que hace un procesador real con su memoria.

En la práctica nadie escribe programas en instrucciones RAM. Escribimos pseudocódigo o Python, y cada línea simple (una asignación, una comparación, una operación aritmética, un acceso a un arreglo) cuesta una cantidad constante de instrucciones. Contemos las operaciones de un algoritmo que suma un arreglo de n números:

```
def suma(A):  
    s = 0                                # 1 asignación  
    for i in range(len(A)):              # n + 1 comparaciones, n incrementos  
        s = s + A[i]                     # 1 acceso, 1 suma, 1 asignación, n veces  
    return s                             # 1
```

Con esa manera de contar, $T(n) = 5n + 3$. Si contamos el acceso y la suma como una sola operación, sale $T(n) = 4n + 3$. Si el compilador guarda $\text{len}(A)$ en un registro, sale otra constante. Las constantes dependen de decisiones arbitrarias sobre cómo contar, y el término n no depende de ninguna. Por eso el análisis de algoritmos descarta las constantes y se queda con la forma en que crece la función.

5. Notación asintótica

Recordamos las definiciones. Sean $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$.

- $f \in O(g)$ si existen $c > 0$ y n_0 tales que $f(n) \leq c \cdot g(n)$ para todo $n \geq n_0$.
- $f \in \Omega(g)$ si existen $c > 0$ y n_0 tales que $f(n) \geq c \cdot g(n)$ para todo $n \geq n_0$.
- $f \in \Theta(g)$ si $f \in O(g)$ y $f \in \Omega(g)$.
- $f \in o(g)$ si para todo $c > 0$ existe n_0 tal que $f(n) < c \cdot g(n)$ para todo $n \geq n_0$. Cuando el límite existe, equivale a $\lim f(n)/g(n) = 0$.
- $f \in \omega(g)$ si $g \in o(f)$, o sea $\lim f(n)/g(n) = \infty$.

NOTACIÓN	SE LEE COMO	EJEMPLO
$f \in O(g)$	$f \leq g$	$5n + 3 \in O(n^2)$
$f \in \Omega(g)$	$f \geq g$	$n^2/100 \in \Omega(n)$
$f \in \Theta(g)$	$f = g$	$5n + 3 \in \Theta(n)$
$f \in o(g)$	$f < g$	$n^{100} \in o(2^n)$
$f \in \omega(g)$	$f > g$	$n \log n \in \omega(n)$

Se escribe $f = O(g)$ aunque sea una pertenencia, y hay que leerlo en una sola dirección: $5n + 3 = O(n^2)$ es cierto y $O(n^2) = 5n + 3$ no significa nada.

Tres jerarquías aparecen todo el tiempo. Para constantes $k > 0$, $\varepsilon > 0$ y $b > 1$:

$$(\log n)^k \in o(n^\varepsilon), \quad n^k \in o(b^n), \quad b^n \in o(n!).$$

Tres errores frecuentes:

1. O es una cota superior. Decir "este algoritmo es $O(n^2)$ " no dice que sea cuadrático: la suma del arreglo también es $O(n^2)$. Cuando queremos decir que el costo crece exactamente como n^2 , escribimos $\Theta(n^2)$.
2. Ω no significa "el mejor caso". El peor caso, el mejor caso y el caso promedio son tres funciones distintas, y a cada una se le pueden dar cotas O , Ω y Θ . Insertion sort cuesta $\Theta(n^2)$ en el peor caso y $\Theta(n)$ en el mejor.
3. La base del logaritmo no importa dentro de O , porque $\log_a n = \log_b n / \log_b a$. La base de una exponencial sí importa: $3^n \notin O(2^n)$.

Salvo que se diga otra cosa, en este curso el costo de un algoritmo es su costo en el peor caso. El costo amortizado aparece en la conferencia 4 y el costo esperado de los algoritmos aleatorios en la última conferencia.

6. Tiempo y memoria

Medimos dos recursos. El tiempo es el número de operaciones y la memoria es el número de celdas que el algoritmo usa además de la entrada.

La memoria nunca supera al tiempo: cada operación toca una cantidad constante de celdas, así que un algoritmo que corre T pasos usa $O(T)$ celdas. Al revés no hay cota, y en la práctica la memoria suele acabarse antes que la paciencia. La subsecuencia común más larga de dos cadenas de longitud n se resuelve

con una tabla de $n \times n$. Con $n = 10^5$ son 10^{10} operaciones simples, del orden de segundos o decenas de segundos en C, y 10^{10} celdas de 4 bytes, 40 GB de memoria que tu computadora no tiene. En la conferencia 10 vamos a ver cómo resolver el mismo problema en tiempo $O(n^2)$ y memoria $O(n)$.

7. Por qué la complejidad importa más cuando la computadora mejora

Supongamos una computadora que hace 10^9 operaciones por segundo y le damos una hora. ¿Cuál es la entrada más grande que resuelve un algoritmo de cada complejidad? ¿Y si la computadora es 10 veces o 1000 veces más rápida?

```
import math

def mayor_n(costo, presupuesto):
    # el mayor n con costo(n) <= presupuesto, por búsqueda binaria
    alto = 2
    while costo(alto) <= presupuesto:
        alto *= 2
    bajo = alto // 2
    while bajo + 1 < alto:
        medio = (bajo + alto) // 2
        if costo(medio) <= presupuesto:
            bajo = medio
        else:
            alto = medio
    return bajo

costos = {
    "n": lambda n: n,
    "n log n": lambda n: n * math.log2(n),
    "n^2": lambda n: n ** 2,
    "n^3": lambda n: n ** 3,
    "2^n": lambda n: 2 ** n,
}

hora = 3600 * 10**9
print(f'{"costo":>8} {"actual":>12} {"10x":>12} {"1000x":>12}')
```

```
for nombre, costo in costos.items():
    fila = [mayor_n(costo, hora * k) for k in (1, 10, 1000)]
    print(f'{"nombre":>8} " + " ".join(f'{"n":>12.3g}" for n in fila))
```

costo	actual	10x	1000x
n	3.6e+12	3.6e+13	3.6e+15
n log n	9.86e+10	9.06e+11	7.8e+13
n^2	1.9e+06	6e+06	6e+07
n^3	1.53e+04	3.3e+04	1.53e+05
2^n	41	45	51

Con una computadora 1000 veces más rápida, el algoritmo lineal resuelve entradas 1000 veces más grandes. El cuadrático, solo $\sqrt{1000} \approx 32$ veces más grandes. El cúbico, 10 veces. El exponencial suma unas 10 unidades al tamaño de entrada, porque $2^{n+10} \approx 1000 \cdot 2^n$. Para un algoritmo exponencial, mil veces más potencia de cómputo compra casi nada.

Ahora mira el problema desde el otro lado. Los datos también crecen. Si en diez años la computadora es 10 veces más rápida y los datos que hay que procesar son 10 veces más grandes, el algoritmo lineal tarda lo mismo que hoy. El cuadrático tarda 10 veces más, porque la entrada creció 10 veces y el costo creció 100. Con el tiempo, un algoritmo cuadrático se vuelve peor, no mejor, y por eso mejorar el algoritmo vale más que esperar por el hardware.

8. El tamaño de la entrada se mide en bits

Volvemos a la primalidad. Hasta ahora medimos el costo en función de n , pero n es el valor de la entrada, no su tamaño. Para escribir n en binario hacen falta $b = \lfloor \log_2 n \rfloor + 1$ bits, y un algoritmo se mide en función del tamaño de su entrada, porque eso es lo que ocupa en memoria y lo que hay que leer.

En función de b , las $\sqrt{n}$ divisiones son $\sqrt{2^b} = 2^{b/2}$. El algoritmo es exponencial en el tamaño de la entrada. Con 10^9 divisiones por segundo:

```
segundos_por_año = 3600 * 24 * 365

for bits in [32, 64, 128, 256, 1024]:
    divisiones = 2 ** (bits // 2)
    segundos = divisiones / 10**9
    if segundos < 60:
        tiempo = f"{segundos:.3g} segundos"
    else:
        tiempo = f"{segundos / segundos_por_año:.3g} años"
    print(f"{bits:>5} bits {float(divisiones):>9.3g} divisiones {tiempo}")
```

32 bits	6.55e+04 divisiones	6.55e-05 segundos
64 bits	4.29e+09 divisiones	4.29 segundos
128 bits	1.84e+19 divisiones	585 años
256 bits	3.4e+38 divisiones	1.08e+22 años
1024 bits	1.34e+154 divisiones	4.25e+137 años

El Sol se apaga en unos $5 \cdot 10^9$ años. Los números de 1024 bits son los que usa la criptografía de clave pública, y para ellos se usa el test de Miller-Rabin, un algoritmo aleatorio polinomial en b . Desde 2002 también se conoce un algoritmo determinista polinomial en b , el de Agrawal, Kayal y Saxena.

Un algoritmo es polinomial si su costo es polinomial en el tamaño de la entrada. Cuando el costo es polinomial en el valor de un número de la entrada pero no en su cantidad de bits, se dice pseudopolinomial. La mochila con su tabla de $O(nW)$ es el ejemplo clásico, y va a ser importante en el Tema 3.

9. Cuánto cuesta una multiplicación

El modelo RAM tiene un problema escondido. Cada celda guarda un entero, sin límite de tamaño, y multiplicar dos enteros cuesta 1. Mira este programa, que eleva al cuadrado el mismo número k veces:

```
import time

x = 2
for k in range(1, 23):
    inicio = time.perf_counter()
    x = x * x
    ms = (time.perf_counter() - inicio) * 1000
    if k >= 14:
        print(f"k = {k:2} bits de x: {x.bit_length():>9,} tiempo: {ms:8.3f} ms")
```

k = 14	bits de x:	16,385	tiempo:	0.044 ms
k = 15	bits de x:	32,769	tiempo:	0.083 ms
k = 16	bits de x:	65,537	tiempo:	0.169 ms
k = 17	bits de x:	131,073	tiempo:	0.260 ms
k = 18	bits de x:	262,145	tiempo:	0.521 ms
k = 19	bits de x:	524,289	tiempo:	1.517 ms

k = 20	bits de x: 1,048,577	tiempo: 3.249 ms
k = 21	bits de x: 2,097,153	tiempo: 7.319 ms
k = 22	bits de x: 4,194,305	tiempo: 33.641 ms

Después de k multiplicaciones, $x = 2^{2^k}$ tiene $2^k + 1$ bits. En el modelo RAM con multiplicación de costo 1, el programa hace k operaciones y construye un número de tamaño exponencial en k . En una computadora real, cada multiplicación tarda más que la anterior. Esto no es solo una curiosidad: Hartmanis y Simon demostraron en 1974 que una RAM con multiplicación de costo unitario resuelve en tiempo polinomial problemas que se consideran mucho más difíciles que los NP-completos del Tema 3.

La solución es limitar el tamaño de las celdas. En el modelo word RAM, cada celda guarda una palabra de w bits, con $w \geq \log_2 n$ para que una palabra alcance para guardar una posición de la entrada. Las operaciones sobre palabras cuestan 1. Es lo que hace un procesador de 64 bits, y es el modelo que usamos de ahora en adelante, casi siempre sin mencionarlo.

Contar la aritmética como $O(1)$ es razonable cuando los números del algoritmo caben en una palabra: índices, contadores, longitudes, pesos acotados. Deja de serlo cuando el algoritmo es numérico y los números crecen: primalidad, máximo común divisor, aritmética modular, enteros grandes, eliminación gaussiana con números exactos. En esos casos contamos operaciones sobre bits. Sobre números de b bits:

OPERACIÓN	COSTO EN OPERACIONES DE BITS
suma y resta	$O(b)$
multiplicación de escuela	$O(b^2)$
Karatsuba	$O(b^{\log_2 3}) \approx O(b^{1.585})$
basada en la transformada de Fourier	$O(b \log b \log \log b)$
Harvey y van der Hoeven (2019)	$O(b \log b)$

La transformada de Fourier es el tema de la conferencia 6. Como ejemplo de análisis por bits, el algoritmo de Euclides sobre números de b bits hace $O(b)$ iteraciones, cada una con una división de costo $O(b^2)$, lo que da $O(b^3)$. Un análisis más fino da $O(b^2)$.

10. El curso

La asignatura tiene tres temas. Cada conferencia toma un problema y recorre el mismo ciclo: proponer un algoritmo, demostrar que es correcto, analizar su costo y, cuando se pueda, demostrar que no se puede hacer mejor.

- Tema 1, fundamentos: 1. Eficiencia y modelos de cómputo. 2. El oficio algorítmico. 3. Correctitud. 4. Análisis amortizado. 5. Cotas mínimas.
- Tema 2, técnicas: 6. Divide y vencerás y recurrencias. 7. Transformada rápida de Fourier. 8. Algoritmos aleatorios: Las Vegas. 9. Golosos y argumentos de intercambio. 10. Matroides. 11. Programación dinámica sobre árboles y subconjuntos. 12. Optimizaciones de programación dinámica. 13. Flujo máximo con Dinic. 14. Flujo máximo con push-relabel. 15. Algoritmos aleatorios: Monte Carlo.
- Tema 3, intratabilidad: 16. P, NP y reducciones. 17. Cook-Levin. 18. Reducciones a grafos. 19. Reducciones a secuencias y números. 20. Algoritmos exactos. 21. Aproximación. 22. TSP métrico y programación lineal. 23. Esquemas de aproximación.
- Cierre: 24. Fronteras del diseño y análisis de algoritmos.

El curso asume la búsqueda binaria, que viste en Programación en primer año, y lo que viste en Estructuras de Datos y Algoritmos: ordenamientos, BFS y DFS, Dijkstra, Bellman-Ford, Floyd-Warshall, árboles recubridores mínimos, union-find, heaps, tablas hash, el Teorema Maestro y la programación dinámica clásica. No asume nada sobre P, NP ni reducciones.

La evaluación tiene cuatro partes:

- Evaluación continua, con los ejercicios para la casa que se orientan después de cada conferencia.
- Parcial 1, al cerrar el Tema 2.
- Parcial 2, al cerrar el Tema 3.
- Examen final escrito, sobre los tres temas.

Puedes usar herramientas de IA para implementar algoritmos, generar casos de prueba y depurar, y tienes que declarar que las usaste. Las demostraciones de correctitud, los análisis de complejidad, las cotas mínimas, las reducciones y las garantías de aproximación o probabilísticas se escriben a mano, en los ejercicios, en los parciales y en el examen.

La bibliografía básica es Introduction to Algorithms de Cormen, Leiserson, Rivest y Stein, y Algorithm Design de Kleinberg y Tardos.

11. Resumen

- Medimos algoritmos contando operaciones en un modelo de cómputo, no en segundos.
- La máquina de Turing define qué es computable, pero cambia los exponentes. Para distinguir n de n^2 usamos el modelo RAM, donde acceder a cualquier celda cuesta 1.
- Las constantes dependen de cómo se cuenta, así que describimos el costo con O , Ω , Θ , o y ω , en el peor caso salvo que se diga otra cosa.
- Medimos tiempo y memoria. La memoria está acotada por el tiempo, y suele ser la que se acaba primero.
- Una computadora más rápida beneficia mucho a los algoritmos lineales y casi nada a los exponenciales, y con datos que crecen, un algoritmo cuadrático empeora con los años.
- El tamaño de la entrada se mide en bits. Probar divisores hasta $\sqrt{n}$ es exponencial en ese tamaño.
- La aritmética cuesta $O(1)$ sobre palabras de $O(\log n)$ bits. En algoritmos numéricos con números que crecen, contamos operaciones de bits.

Ejercicios

1. Demuestra con la definición, dando c y n_0 , que $3n^2 + 10n \log_2 n + 7 \in \Theta(n^2)$.
2. Decide si cada afirmación es verdadera o falsa y demuéstralo: $2^{n+1} \in O(2^n)$; $2^{2n} \in O(2^n)$; $\log_2(n!) \in \Theta(n \log n)$; $n^{\log_2 n} \in O(2^n)$.
3. Da dos funciones f y g tales que $f \notin O(g)$ y $g \notin O(f)$.
4. Una computadora 100 veces más rápida, ¿cuánto aumenta el tamaño de entrada que resuelve en una hora un algoritmo de costo $n \log_2 n$? Estima el factor y compruébalo modificando el programa de la sección 7.
5. Demuestra que un algoritmo que corre $T(n)$ pasos en el modelo RAM usa $O(T(n))$ celdas de memoria además de la entrada.
6. El siguiente algoritmo calcula el n -ésimo número de Fibonacci con n sumas.

```
def fibonacci(n):  
    a, b = 0, 1  
    for _ in range(n):  
        a, b = b, a + b  
    return a
```

Sabiendo que F_n tiene $\Theta(n)$ bits, calcula su costo en operaciones de bits. Después exprésalo en función del tamaño de la entrada $b = \lfloor \log_2 n \rfloor + 1$. ¿Es polinomial?

7. Diseña una máquina de Turing de una cinta que sume 1 a un número escrito en binario, con el cabezal empezando en el bit menos significativo y volviendo a él al terminar. ¿Cuántos pasos da en el peor caso? Si se aplica m veces seguidas empezando desde 0, ¿cuántos pasos da en total? El ejercicio 4 del resumen del Tema 1 vuelve sobre esta pregunta con las herramientas de la conferencia 4.